
JAL240I
PRINCIPLES OF ARTIFICIAL INTELLIGENCE

UNIT IV
KNOWLEDGE AND REASONING

UNIT IV - KNOWLEDGE AND REASONING

SYLLABUS

First-Order Logic – Knowledge Representation - Syntax and Semantics of First-Order Logic - Using First-Order Logic - Knowledge Engineering in First-Order Logic

Inference in First-Order Logic - Propositional vs. First-Order Inference - Unification and First-Order Inference - Forward Chaining - Backward Chaining - Quantifying Uncertainty - Bayes' Rule and Its Use - Naive Bayes Model.

2

FIRST-ORDER LOGIC

3

FIRST-ORDER LOGIC

- Knowledge base (KB) → the central component of a knowledge-based agent → it is a set of sentences → initially contain some background knowledge.
- Each sentence is expressed in a language called a **knowledge representation language** → represents some assertion about the world.
- The agent maintains a KB → has a way to add new sentences to the knowledge base (TELL operation) and a way to query what is known (ASK operation) → ASK will be followed by TELL
- Both operations may involve inference → means, deriving new sentences from old.
- **Propositional logic** → a declarative language because its semantics is based on a truth relation between sentences and possible worlds → uses simple statement s that can be T or F → → can be used to deal with partial information, using **disjunction** ($\vee$), **conjunction** ($\wedge$) and **negation** ($\neg$) → eg. “it is raining”, “the road is wet”.
- **First-order logic** → can concisely represent knowledge much more than propositional logic.

KNOWLEDGE REPRESENTATION

- THE LANGUAGE OF THOUGHT
- COMBINING THE BEST OF FORMAL AND NATURAL LANGUAGES

5

KNOWLEDGE REPRESENTATION THE LANGUAGE OF THOUGHT

- **First-Order Logic (FOL)**, also known as **First-Order Predicate Logic** → a formal system used to express statements with quantifiable variables, predicates, and logical connectives → extends **Propositional Logic** by allowing more expressive representations → allows AI to represent and reason about objects, properties, and relationships between them.
- Used in AI, databases, theorem proving, and formal reasoning.
- **Compositionality** → In a compositional language, the meaning of a sentence is a function of the meaning of its parts.
- Natural languages are highly expressive and traditionally viewed as declarative knowledge representation tools. However, modern perspectives emphasize their role in communication, where meaning depends on both language and context. Ambiguity in natural language makes it challenging to use for formal knowledge representation.
- The Sapir-Whorf hypothesis suggests that language influences thought, but evidence for major cognitive differences is weak. Studies show that while language affects perception (e.g., gendered nouns influencing adjective choices, directional language affecting navigation skills), it does not fundamentally alter worldviews.

KNOWLEDGE REPRESENTATION

THE LANGUAGE OF THOUGHT

- In a logical reasoning system that uses conjunctive normal form (CNF), the linguistic forms “ $\neg(A \vee B)$ ” and “ $\neg A \wedge \neg B$ ” are the same $\rightarrow$ the two sentences are stored as the same canonical CNF form.
- It is starting to become possible to do something similar with the human brain.
- Experiments using fMRI (functional magnetic resonance imaging) suggest common neural representations of words across individuals, hinting at shared cognitive processing.
- However, the efficiency of reasoning and learning depends on how knowledge is represented, with simpler representations often being more effective. Language inevitably influences thought, especially in learning systems where choosing the most concise theory is advantageous.

7

KNOWLEDGE REPRESENTATION - COMBINING

THE BEST OF FORMAL AND NATURAL LANGUAGES

- Foundation of propositional logic $\rightarrow$ a declarative, compositional semantics that is context-independent and unambiguous.
- The elements in the syntax of natural language:
 - Objects $\rightarrow$ nouns $\rightarrow$ eg. people, houses, numbers, theories, Ronald McDonald, colors, baseball games, wars, centuries
 - Relations $\rightarrow$ verbs + adjectives / adverbs $\rightarrow$ eg. unary relations or properties such as red, round, bogus, prime, multistoried, or more general n-ary relations such as brother of, bigger than, inside, part of, has color, occurred after, owns, comes between.
 - Functions $\rightarrow$ relations in which there is only one “value” for a given “input” $\rightarrow$ eg. father of, best friend, third inning of, one more than, beginning of.
- The language of first-order logic has syntax and semantics built around objects and relations.

8

KNOWLEDGE REPRESENTATION - COMBINING THE BEST OF FORMAL AND NATURAL LANGUAGES

Key Components of First-Order Logic

- Constants (Objects) – Represent specific things - Example: `John`, `Apple`, `Earth`
- Predicates (Properties & Relations)** – Describe relationships between objects - Example: `Loves(John, Mary)` → "John loves Mary" - `IsHuman(John)` → "John is a human"
- Variables (General Objects)** – Represent any object in a domain - Example: `x, y, z` (can represent any person, number, etc.)
- Quantifiers (Generalization & Existence)
 - Universal Quantifier ($\forall$) → "For all" - Example: $\forall x \text{ IsHuman}(x) \rightarrow \text{Mortal}(x)$ → "All humans are mortal"
 - Existential Quantifier ($\exists$) → "There exists" - Example: $\exists x \text{ Loves}(x, \text{Mary})$ → "There exists someone who loves Mary"

9

KNOWLEDGE REPRESENTATION - COMBINING THE BEST OF FORMAL AND NATURAL LANGUAGES

How is First-Order Logic Used in AI?

- **1. Expert Systems** – AI derives conclusions from general rules (e.g., medical diagnosis, legal reasoning).
- **2. Natural Language Processing (NLP)** – Helps AI understand sentence meanings based on subject-predicate structure.
- **3. Knowledge Representation** – AI stores and retrieves structured facts (e.g., "All birds can fly except penguins").
- **4. Automated Theorem Proving** – AI proves mathematical and logical statements using FOL-based inference

FOL Better vs Propositional Logic

- Can express relationships between objects
- Allows generalization using variables & quantifiers
- More applicable in AI for real-world reasoning
- More computationally complex than Propositional Logic
- Difficult to implement in large-scale AI systems

10

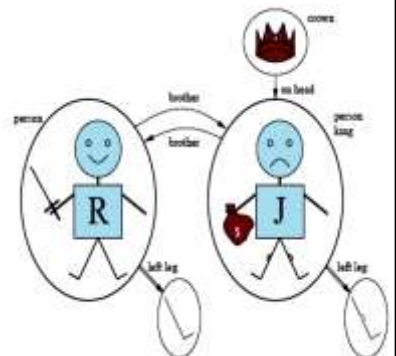
SYNTAX AND SEMANTICS OF FIRST-ORDER LOGIC

- MODELS FOR FIRST-ORDER LOGIC
- SYMBOLS AND INTERPRETATIONS
- TERMS
- ATOMIC SENTENCES
- COMPLEX SENTENCES
- QUANTIFIERS
- EQUALITY
- DATABASE SEMANTICS

11

MODELS FOR FIRST-ORDER LOGIC

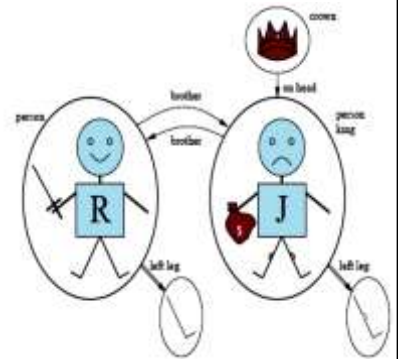
- The **domain** of a model is the set of objects or domain elements it contains → must be non-empty → every possible world must contain at least one object.
- EG. five objects: *Richard the Lionheart*, King of England from 1189 to 1199, his younger brother, *the evil King John*, who ruled from 1199 to 1215; the *left legs of Richard and John*; and a *crown*.
- The objects in the model may be related in various ways.
- A **tuple** is a collection of objects arranged in a fixed order and is written with angle brackets surrounding the objects.
- A **relation** → the set of tuples of objects that are related.
- Eg. $\{ \langle \text{Richard the Lionheart, King John} \rangle, \langle \text{King John, Richard the Lionheart} \rangle \}$



12

MODELS FOR FIRST-ORDER LOGIC

- Functions → used to represent certain kinds of relationships → a given object must be related to exactly one object in this way.
- For example, each person has one left leg, so the model has a unary “left leg” function → a mapping from a one-element tuple to an object.
- Eg. $\langle \text{Richard the Lionheart} \rangle \rightarrow \text{Richard's left leg}$
 $\langle \text{King John} \rangle \rightarrow \text{John's left leg}$.
- The model has → two binary relations (brother and on-head), three unary relations (person, king, and crown) and one unary function (left-leg).



SYMBOLS AND INTERPRETATIONS

- The basic syntactic elements of FOL → are the symbols for objects, relations, and functions.
- 3 kinds of symbols:
 1. Constant symbols → stand for objects → eg. Richard and John
 2. Predicate symbols → stand for relations → eg. Brother, OnHead, Person, King, and Crown
 3. Function symbols → stand for functions → LeftLeg
- These symbols will begin with uppercase letters.
- Each predicate and function symbol has a fixed number of arguments → **arity**.

SYMBOLS AND INTERPRETATIONS

$Sentence \rightarrow AtomicSentence \mid ComplexSentence$
 $AtomicSentence \rightarrow Predicate \mid Predicate(Term, \dots) \mid Term = Term$
 $ComplexSentence \rightarrow (Sentence)$
 $\mid \neg Sentence$
 $\mid Sentence \wedge Sentence$
 $\mid Sentence \vee Sentence$
 $\mid Sentence \Rightarrow Sentence$
 $\mid Sentence \Leftrightarrow Sentence$
 $\mid Quantifier Variable, \dots Sentence$

$Term \rightarrow Function(Term, \dots)$
 $\mid Constant$
 $\mid Variable$
 $Quantifier \rightarrow \forall \mid \exists$
 $Constant \rightarrow A \mid X_1 \mid John \mid \dots$
 $Variable \rightarrow a \mid x \mid s \mid \dots$
 $Predicate \rightarrow True \mid False \mid After \mid Loves \mid Raining \mid \dots$
 $Function \rightarrow Mother \mid LeftLeg \mid \dots$
OPERATOR PRECEDENCE : $\neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow$

TERMS

- A term $\rightarrow$ a logical expression that refers to an object.
- Constant symbols are terms $\rightarrow$ Eg. King John, . King John's left leg $\rightarrow$ in the 2nd eg. instead of such a complex constant symbol, use function symbol LeftLeg(John).
- Semantics of terms $\rightarrow$ a term $f(t_1, \dots, t_n) \rightarrow f$ refers to a function and arguments refer to objects in the domain $\rightarrow$ eg. LeftLeg(John)

ATOMIC SENTENCES

- **Terms** → refer to objects.
- **Predicate symbols** → available for referring to relations
- Terms & predicates can be combined to make atomic sentences → to form **facts**.
- An atomic sentence → formed from a predicate symbol optionally followed by a parenthesized list of terms → $P(x, y)$ is read as “x is a P of y.”
- Eg. $\text{Brother}(\text{Richard}, \text{John})$ → interpreted as Richard is the brother of John.
- An atomic sentence with complex terms → $P(Q(x), R(y))$ read is Q(x) is P to / of R(y)
- Eg. $\text{Married}(\text{Father}(\text{Richard}), \text{Mother}(\text{John}))$ → interpreted as Richard’s father is married to John’s mother.
- An atomic sentence is true in a given model if the relation referred to by the predicate symbol holds among the objects referred to by the arguments.

17

COMPLEX SENTENCES

- Logical connectives are used to construct more complex sentences.
- Eg.
- $\neg \text{Brother}(\text{LeftLeg}(\text{Richard}), \text{John})$.
- $\text{Brother}(\text{Richard}, \text{John}) \wedge \text{Brother}(\text{John}, \text{Richard})$.
- $\text{King}(\text{Richard}) \vee \text{King}(\text{John})$.
- $\neg \text{King}(\text{Richard}) \Rightarrow \text{King}(\text{John})$.

18

QUANTIFIERS

- Quantifiers → Used to express properties of entire collections of objects, instead of enumerating the objects by name.
- FOL contains two standard quantifiers:
 1. Universal quantifiers $\forall$
 2. Existential quantifiers $\exists$
- Universal quantifiers $\forall \rightarrow$ “for all” $\rightarrow$ eg. $\forall x \text{ King}(x) \Rightarrow \text{Person}(x) \rightarrow$ “All kings are persons” (or) “For all x, if x is a king, then x is a person.”
- Existential quantifiers $\exists \rightarrow$ “for some x” $\rightarrow \exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John}). \rightarrow$ John has a crown on his head.

19

QUANTIFIERS

- Universal quantifiers $\forall \rightarrow$ pronounced as “for all”
- $\forall x P \rightarrow$ a logical sentence $\rightarrow$ means P is true for every object x.
- eg. $\forall x \text{ King}(x) \Rightarrow \text{Person}(x) \rightarrow$ “All kings are persons” (or) “For all x, if x is a king, then x is a person.”
- Here symbol $\Rightarrow$ is termed as implication
- Here symbol x is called a variable $\rightarrow$ lower case.
- A term with no variables is called a ground term $\rightarrow$ eg. John.

20

QUANTIFIERS

- Existential quantifier $\exists \rightarrow$ Pronounced as “There exists an x such that ...” or “For some x ...”.
- Statement about some object without naming it $\rightarrow$ done using an existential quantifier.
- $\exists x P \rightarrow$ says that P is true for at least one object x .
- Eg. $\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$.
- This can be checked against the following from which one will be true:
 - Richard the Lionheart is a crown $\wedge$ Richard the Lionheart is on John's head. $\rightarrow I$
 - John's left leg is a crown $\wedge$ John's left leg is on John's head
 - The crown is a crown $\wedge$ the crown is on John's head.
- I is similar to Richard the Lionheart is a crown $\Rightarrow$ Richard the Lionheart is on John's head.
- An implication is true if both premise and conclusion are true, or if its premise is false.

QUANTIFIERS

- Nested quantifier $\rightarrow$ expressing more complex sentences using multiple quantifiers.
- Eg. $\forall x \forall y \text{Brother}(x, y) \Rightarrow \text{Sibling}(x, y) \rightarrow$ “Brothers are siblings”.
- $\forall x, y \text{Sibling}(x, y) \Leftrightarrow \text{Sibling}(y, x) \rightarrow$ gives a symmetric relationship
- $\forall x \exists y \text{Loves}(x, y) \rightarrow$ for all x there exists some $y \rightarrow$ “Everybody loves somebody”
- $\exists x \forall y \text{Loves}(x, y) \rightarrow$ there exist some x for every $y \rightarrow$ “There is someone who is loved by everyone”

QUANTIFIERS

- Demorgan's rules for quantified and unquantified sentences

$$\neg \exists x P \equiv \forall x \neg P$$

$$\neg \forall x P \equiv \exists x \neg P$$

$$\forall x P \equiv \neg \exists x \neg P$$

$$\exists x P \equiv \neg \forall x \neg P$$

$$\neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q$$

$$P \wedge Q \equiv \neg(\neg P \vee \neg Q)$$

$$P \vee Q \equiv \neg(\neg P \wedge \neg Q)$$

23

QUANTIFIERS

Equality

- Equality symbol used to signify that two terms refer to the same object.
- Eg. $\text{Father}(\text{John}) = \text{Henry}$ $\rightarrow$ means $\text{Father}(\text{John})$ and the object referred to by Henry are the same.

Database semantics

- $\text{Brother}(\text{John}, \text{Richard}) \wedge \text{Brother}(\text{Geoffrey}, \text{Richard}) \rightarrow$ Richard has two brothers, John and Geoffrey
- In database systems the following assumptions hold:
 - $\rightarrow$ Every constant symbol refer to a distinct object $\rightarrow$ **the unique-names assumption.**
 - $\rightarrow$ Atomic sentences not known to be true are in fact false $\rightarrow$ **the closed world assumption.**
 - $\rightarrow$ Each model contains no more domain elements than those named by the constant symbols $\rightarrow$ **domain closure**

24

USING FIRST-ORDER LOGIC

- **ASSERTIONS AND QUERIES IN FIRST-ORDER LOGIC**
- **THE KINSHIP DOMAIN**
- **THE WUMPUS WORLD**

25

USING FIRST-ORDER LOGIC

- Domains → In knowledge representation, is just some part of the world about which we wish to express some knowledge.
- TELL/ASK interface for first-order knowledge bases.
- Domains of family relationships, numbers, sets, and lists
- The wumpus world.

26

ASSERTIONS

- Assertions → Sentences added to a knowledge base using TELL.
- Add sentences to knowledge base using TELL. – eg.:
 - John is a king → TELL(KB, King(John)).
 - John is a person → TELL(KB, Person(John)).
 - Richard is a person → TELL(KB, Person(Richard)).
 - All kings are persons → TELL(KB, $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$).
- Queries (or goals) → Questions asked to the knowledge base with ASK
- Ask questions of the knowledge base using ASK – eg.
 - ASK(KB, King(John)) → returns True
 - ASK(KB, Person(John)) → returns True
 - ASK(KB, $\exists x \text{ Person}(x)$) → returns True
 - $\exists x \text{ King}(x)$ → returns True
- The last answer is called as called a substitution or binding list.

27

THE KINSHIP DOMAIN

- Kinship domain → domain of family relationships.
- Objects in our domain are people.
- Unary predicates → Male and Female, etc.
- Kinship relations → parenthood, brotherhood, marriage, and so on → represented by binary predicates: Parent, Sibling, Brother, Sister, Child, Daughter, Son, Spouse, Wife, Husband, Grandparent, Grandchild, Cousin, Aunt, and Uncle.
- Functions → used for Mother and Father, etc.
- Uses axiom or kinship axioms → definitions → have the following form
$$\forall x, y \text{ P}(x, y) \Leftrightarrow \dots$$
- Some provide more general information about certain predicates without constituting a definition.
- Some predicates have no complete definition because we do not know enough to characterize them fully.

28

THE KINSHIP DOMAIN

- Eg.
- One's mother is one's parent who is female $\rightarrow \forall m, c \text{ Mother}(c)=m \Leftrightarrow \text{Female}(m) \wedge \text{Parent}(m, c)$.
- One's husband is one's male spouse $\rightarrow \forall w, h \text{ Husband}(h, w) \Leftrightarrow \text{Male}(h) \wedge \text{Spouse}(h, w)$.
- Parent and child are inverse relations $\rightarrow \forall p, c \text{ Parent}(p, c) \Leftrightarrow \text{Child}(c, p)$.
- A grandparent is a parent of one's parent $\rightarrow \forall g, c \text{ Grandparent}(g, c) \Leftrightarrow \exists p \text{ Parent}(g, p) \wedge \text{Parent}(p, c)$.
- A sibling is another child of one's parent $\rightarrow \forall x, y \text{ Sibling}(x, y) \Leftrightarrow x \neq y \wedge \exists p \text{ Parent}(p, x) \wedge \text{Parent}(p, y)$.

29

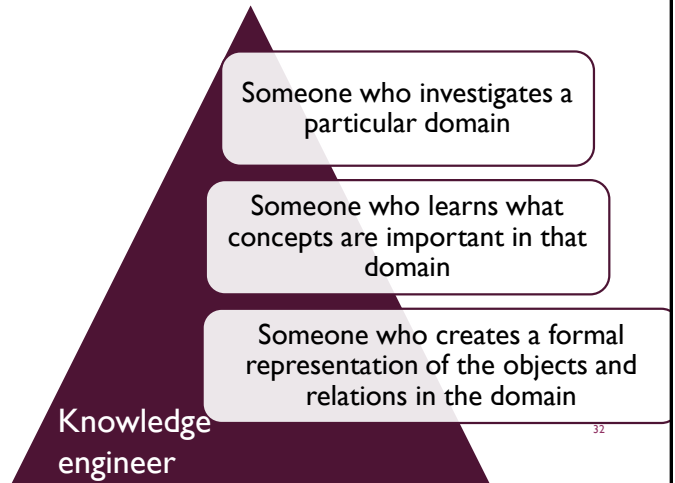
KNOWLEDGE ENGINEERING IN FIRST-ORDER LOGIC

- THE KNOWLEDGE ENGINEERING PROCESS
- THE ELECTRONIC CIRCUITS DOMAIN

31

KNOWLEDGE ENGINEERING IN FIRST-ORDER LOGIC

- Process of knowledge-base construction → knowledge engineering process.
- A knowledge engineer →
- The knowledge engineering process is suitable for developing special-purpose knowledge bases whose domain is carefully circumscribed and whose range of queries is known in advance.
- General-purpose knowledge bases → support tasks such as natural language understanding.



THE KNOWLEDGE ENGINEERING PROCESS

1. Identify the questions



2. Assemble the relevant knowledge



3. Decide on a vocabulary of predicates, functions, and constants



4. Encode general knowledge about the domain



5. Encode a description of the problem instance



6. Pose queries to the inference procedure and get answers



7. Debug and evaluate the knowledge base

33

THE KNOWLEDGE ENGINEERING PROCESS

1. Identify the Questions

- The knowledge engineer must delineate the range of questions that the knowledge base will support and the kinds of facts that will be available for each specific problem instance.
- The task will determine what knowledge must be represented in order to connect problem instances to answers.
- This step is analogous to the PEAS process for designing agents.

2. Assemble the Relevant Knowledge

- Collect domain knowledge from experts or other sources. At this stage, focus on understanding the domain rather than formalizing it.
- Distinguishing relevant information is straightforward in artificial domains but challenging in real-world scenarios.

34

THE KNOWLEDGE ENGINEERING PROCESS

3. Decide on a Vocabulary

- Choose predicates, functions, and constants to represent domain concepts formally.
- These choices form the ontology → the set of terms representing what exists in the domain.
- Good design choices here influence clarity and correctness.

4. Encode General Knowledge

- Write logical axioms that define the meaning of vocabulary terms.
- This helps ensure consistency and may uncover gaps or misunderstandings in earlier steps.

35

THE KNOWLEDGE ENGINEERING PROCESS

5. Encode the Problem Instance

- Describe specific scenarios using atomic sentences derived from the ontology.
- In agents, this step corresponds to receiving sensor data.

6. Pose Queries and Get Answers

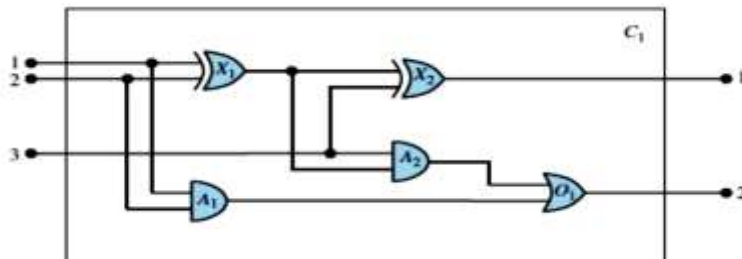
- Use inference to derive answers from the encoded knowledge and problem instances, removing the need for task-specific algorithms.

7. Debug and Evaluate the Knowledge Base

- Check whether the system gives correct and expected answers.
- Common issues include missing or incorrect axioms.
- In debugging failed reasoning chains are traced and flawed or incomplete knowledge is corrected.

THE ELECTRONIC CIRCUITS DOMAIN

- Knowledge engineering process to design the following circuit:



A digital circuit C_1 , purporting to be a one-bit full adder. The first two inputs are the two bits to be added, and the third input is a carry bit. The first output is the sum, and the second output is a carry bit for the next adder. The circuit contains two XOR gates, two AND gates, and one OR gate.

37

THE ELECTRONIC CIRCUITS DOMAIN

1. Identify the Questions

- Reasoning tasks associated with digital circuits → more detailed levels of analysis, including those related to timing delays, circuit area, circuit's structure, power consumption, production cost, etc. And finally circuit's functionality.
- Example questions:
- Does the circuit actually add properly?
- If all the inputs are high, what is the output of gate A2?
- What are all the gates connected to the first input terminal?
- Does the circuit contain feedback loops?
- With additional knowledge, these questions can be answered.

38

THE ELECTRONIC CIRCUITS DOMAIN

2. Assemble the relevant knowledge

- About digital circuits → composed of wires and gates - signals flow along wires to the input terminals of gates, and each gate produces a signal on the output terminal that flows along another wire - AND, OR, and XOR gates have two input terminals, and NOT gates have one – All gates have one output terminal.
- To reason about functionality and connectivity
 - Relevant knowledge → the connections between terminals
 - Irrelevant knowledge → size, shape, color, or cost of the various components
- To verify designs
 - For debugging faulty circuits → include the wires
 - For resolving timing faults → include gate delays.
 - For designing a profitable product → include cost of the circuit and its speed.

39

THE ELECTRONIC CIRCUITS DOMAIN

3. Decide on a vocabulary

- Decide on circuits, terminals, signals, and gates and predicates and functions to represent them
- Gate $\rightarrow$ represented as $\text{Gate}(X_i)$
- Behavior of each gate $\rightarrow$ represented as $\text{Type}(X_i)=\text{XOR}, \text{Type}(X_i)=\text{AND}$, etc.
- Circuits $\rightarrow$ represented as $\text{Circuit}(CI)$
- Terminals $\rightarrow$ identified as $\text{Terminal}(x)$
- 1st input terminal for $XI \rightarrow \text{In}(1, X_i)$ 2nd input terminal for $XI \rightarrow \text{In}(2, X_i)$
- Output terminal $\rightarrow \text{Out}(n, c)$
- Circuit c has i input and j output terminals $\rightarrow$ rep as $\text{Arity}(c, i, j)$
- Connectivity between gates $\rightarrow \text{Connected}(\text{Out}(1, X_i), \text{In}(1, X_2))$
- To check if a signal is on or off $\rightarrow \text{On}(t) \rightarrow$ true or false

40

THE ELECTRONIC CIRCUITS DOMAIN

4. Encode general knowledge of the domain

- A good ontology $\rightarrow$ uses few general rules
 1. If two terminals are connected, then they have the same signal:

$$\forall t_1, t_2 \text{ Terminal}(t_1) \wedge \text{Terminal}(t_2) \wedge \text{Connected}(t_1, t_2) \Rightarrow \text{Signal}(t_1) = \text{Signal}(t_2) .$$
 2. The signal at every terminal is either 1 or 0:

$$\forall t \text{ Terminal}(t) \Rightarrow \text{Signal}(t) = 1 \vee \text{Signal}(t) = 0 .$$
 3. Connected is commutative:

$$\forall t_1, t_2 \text{ Connected}(t_1, t_2) \Leftrightarrow \text{Connected}(t_2, t_1) .$$
 4. There are four types of gates:

$$\forall g \text{ Gate}(g) \wedge k = \text{Type}(g) \Rightarrow k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR} \vee k = \text{NOT} .$$
 5. An AND gate's output is 0 if and only if any of its inputs is 0:

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{AND} \Rightarrow \text{Signal}(\text{Out}(1, g)) = 0 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 0 .$$
 6. An OR gate's output is 1 if and only if any of its inputs is 1:

$$\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{OR} \Rightarrow \text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \exists n \text{ Signal}(\text{In}(n, g)) = 1 .$$

41

THE ELECTRONIC CIRCUITS DOMAIN

4. Encode general knowledge of the domain

7. An XOR gate's output is 1 if and only if its inputs are different:
 $\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{XOR} \Rightarrow$
 $\text{Signal}(\text{Out}(1, g)) = 1 \Leftrightarrow \text{Signal}(\text{In}(1, g)) \neq \text{Signal}(\text{In}(2, g)) .$
8. A NOT gate's output is different from its input:
 $\forall g \text{ Gate}(g) \wedge \text{Type}(g) = \text{NOT} \Rightarrow$
 $\text{Signal}(\text{Out}(1, g)) \neq \text{Signal}(\text{In}(1, g)) .$
9. The gates (except for NOT) have two inputs and one output.
 $\forall g \text{ Gate}(g) \wedge \text{Type}(g) \neq \text{NOT} \Rightarrow \text{Arity}(g, 1, 1) .$
 $\forall g \text{ Gate}(g) \wedge k = \text{Type}(g) \wedge (k = \text{AND} \vee k = \text{OR} \vee k = \text{XOR}) \Rightarrow$
 $\text{Arity}(g, 2, 1)$
10. A circuit has terminals, up to its input and output arity, and nothing beyond its arity:
 $\forall c, i, j \text{ Circuit}(c) \wedge \text{Arity}(c, i, j) \Rightarrow$
 $\forall n (n \leq i \Rightarrow \text{Terminal}(\text{In}(n, c))) \wedge (n > i \Rightarrow \text{In}(n, c) = \text{Nothing}) \wedge$
 $\forall n (n \leq j \Rightarrow \text{Terminal}(\text{Out}(n, c))) \wedge (n > j \Rightarrow \text{Out}(n, c) = \text{Nothing})$
11. Gates, terminals, and signals are all distinct.
 $\forall g, t, s \text{ Gate}(g) \wedge \text{Terminal}(t) \wedge \text{Signal}(s) \Rightarrow$
 $g \neq t \wedge g \neq s \wedge t \neq s .$
12. Gates are circuits.
 $\forall g \text{ Gate}(g) \Rightarrow \text{Circuit}(g)$

42

THE ELECTRONIC CIRCUITS DOMAIN

5. Encode the problem instance

Circuit and its component gates

Connections between them

$\text{Circuit}(C_1) \wedge \text{Arity}(C_1, 3, 2)$
 $\text{Gate}(X_1) \wedge \text{Type}(X_1) = \text{XOR}$
 $\text{Gate}(X_2) \wedge \text{Type}(X_2) = \text{XOR}$
 $\text{Gate}(A_1) \wedge \text{Type}(A_1) = \text{AND}$
 $\text{Gate}(A_2) \wedge \text{Type}(A_2) = \text{AND}$
 $\text{Gate}(O_1) \wedge \text{Type}(O_1) = \text{OR} .$

$\text{Connected}(\text{Out}(1, X_1), \text{In}(1, X_2))$ $\text{Connected}(\text{In}(1, C_1), \text{In}(1, X_1))$
 $\text{Connected}(\text{Out}(1, X_1), \text{In}(2, A_2))$ $\text{Connected}(\text{In}(1, C_1), \text{In}(1, A_1))$
 $\text{Connected}(\text{Out}(1, A_2), \text{In}(1, O_1))$ $\text{Connected}(\text{In}(2, C_1), \text{In}(2, X_1))$
 $\text{Connected}(\text{Out}(1, A_1), \text{In}(2, O_1))$ $\text{Connected}(\text{In}(2, C_1), \text{In}(2, A_1))$
 $\text{Connected}(\text{Out}(1, X_2), \text{Out}(1, C_1))$ $\text{Connected}(\text{In}(3, C_1), \text{In}(2, X_2))$
 $\text{Connected}(\text{Out}(1, O_1), \text{Out}(2, C_1))$ $\text{Connected}(\text{In}(3, C_1), \text{In}(1, A_2))$

THE ELECTRONIC CIRCUITS DOMAIN

6. Pose queries to the inference procedure

- What combinations of inputs would cause the first output of C1 (the sum bit) to be 0 and the second output of C1 (the carry bit) to be 1?

$$\exists i_1, i_2, i_3 \text{ Signal(In}(1, C_1)) = i_1 \wedge \text{Signal(In}(2, C_1)) = i_2 \wedge \text{Signal(In}(3, C_1)) = i_3 \\ \wedge \text{Signal(Out}(1, C_1)) = 0 \wedge \text{Signal(Out}(2, C_1)) = 1.$$

- The answers are substitutions for the variables i_1 , i_2 , and i_3 based on the knowledge base

$$\{i_1/1, i_2/1, i_3/0\} \quad \{i_1/1, i_2/0, i_3/1\} \quad \{i_1/0, i_2/1, i_3/1\}.$$

44

THE ELECTRONIC CIRCUITS DOMAIN

7. Debug the knowledge base

- Suppose it is found that the system is unable to prove any outputs for the circuit, except for 000 and 110, the following query can be asked

$$\exists i_1, i_2, o \text{ Signal(In}(1, C_1)) = i_1 \wedge \text{Signal(In}(2, C_1)) = i_2 \wedge \text{Signal(Out}(1, X_1)) = o$$

- It reveals that no outputs are known at X_1 for the input cases 10 and 0.
- Then we look at the axiom for XOR gates, as applied to X_1 :

$$\text{Signal(Out}(1, X_1)) = 1 \Leftrightarrow \text{Signal(In}(1, X_1)) \neq \text{Signal(In}(2, X_1))$$

- If the inputs are known to be, say, 1 and 0, then this reduces to

$$\text{Signal(Out}(1, X_1)) = 1 \Leftrightarrow 1 \neq 0$$

- Now the problem is apparent: the system is unable to infer that $\text{Signal(Out}(1, X_1)) = 1$

45

FORWARD & BACKWARD CHAINING

46

FORWARD CHAINING

- Rules of the form **Antecedent $\Rightarrow$ Consequent**
- **First-order definite clauses** $\rightarrow$ disjunctions of literals of which exactly one is positive $\rightarrow$ means a definite clause is either atomic, or is an implication whose antecedent is a conjunction of positive literals and whose consequent is a single positive literal $\rightarrow$ eg. $\text{King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$
- Existential quantifiers are not allowed; universal quantifiers are implicit.
- Datalog $\rightarrow$ a language consisting of first-order definite clauses with no function symbols.
- Forward chaining inference algorithm $\rightarrow$ Starting from the known facts, it triggers all the rules whose premises are satisfied, adding their conclusions to the known facts $\rightarrow$ the process repeats until the query is answered (assuming that just one answer is required) or no new facts are added.
- Eg. $\text{Likes}(x, \text{IceCream})$ and $\text{Likes}(y, \text{IceCream})$ are renamings of each other.

47

BACKWARD CHAINING

- Backward chaining involves reasoning from an end goal, working backward to determine the sequence of events or steps that could lead to that goal.
- It is a type of reasoning that enables an AI system to ascertain the root cause of a problem or the series of steps required to achieve a specific objective.

48